

Graphs: BFS and DFS Problems

A Problem-Solving Companion to Striver's A2Z and SDE Sheets

Maruri Saikiran

June 30, 2026

Contents

1	Graphs: BFS and DFS Problems	1
1.1	One Traversal, Several Extra Questions	1
1.2	Detect Cycle in an Undirected Graph	2
1.3	Bipartite Graph Check	4
1.4	Surrounded Regions	6
1.5	Number of Enclaves	8
1.6	Word Ladder I	10
1.7	Word Ladder II	11

Chapter 1

Graphs: BFS and DFS Problems

Every problem in this chapter is still, underneath, a single traversal – but each one asks the traversal to carry one extra piece of bookkeeping along for the ride: a parent to avoid false positives, a color to split the graph in two, a boundary flag, or a neighbor relation defined by edit distance rather than an explicit edge list.

1.1 One Traversal, Several Extra Questions

The Graph Basics chapter established DFS and BFS as the two ways to walk a graph. This chapter keeps that same walk and asks it to answer one additional question *while* walking:

What Each Problem Adds to the Base Traversal

- **Detect Cycle in an Undirected Graph (Section 1.2):** carry the **parent** node along; reaching an already-visited node that is *not* the parent means a cycle exists.
- **Bipartite Check (Section 1.3):** carry a **color** (alternating 0/1) along; a neighbor already colored the *same* color as the current node means the graph cannot be 2-colored.
- **Surrounded Regions (Section 1.4) and Number of Enclaves (Section 1.5):** run the traversal **only from the grid's boundary** first, to mark every region that escapes the edge – everything left unmarked is, by elimination, fully enclosed.
- **Word Ladder I and II (Sections 1.6–1.7):** the graph is entirely **implicit** – two words are “neighbors” if they differ by exactly one character – so BFS explores generated candidate words instead of a stored adjacency list.

1.2 Detect Cycle in an Undirected Graph

Problem Statement. Given an undirected graph, determine whether it contains a **cycle**.

Pattern Recognition

“Does a cycle exist” is a traversal that carries one extra piece of state along: the **parent** node (the node the current node was reached from). Reaching an already-visited neighbor that is **not** the parent means there are two distinct paths to that neighbor – which is exactly what a cycle is.

Why checking against the parent specifically is necessary

In an *undirected* graph, every edge (u, v) is stored twice: once as $u \rightarrow v$ and once as $v \rightarrow u$. So when DFS or BFS moves from u to v , v ’s neighbor list contains u again – without excluding the parent, this single edge would be misreported as a cycle every time. Excluding only the *immediate* parent (not all previously visited nodes in general) is correct because any *other* already-visited neighbor reached again is a genuinely different path back to that node, while the parent connection is simply the edge just traversed in reverse.

Approach 1: DFS

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool dfsHasCycle(int node, int parent, vector<vector<int>>& adj, vector<bool>&
    visited) {
5      visited[node] = true;
6
7      for (int neighbor : adj[node]) {
8          if (!visited[neighbor]) {
9              if (dfsHasCycle(neighbor, node, adj, visited)) return true;
10             } else if (neighbor != parent) {
11                 return true; // visited, and not the edge we just came from
12             }
13         }
14         return false;
15     }
16
17     bool hasCycleDFS(int n, vector<vector<int>>& adj) {
18         vector<bool> visited(n, false);
19         for (int i = 0; i < n; i++) {
20             if (!visited[i] && dfsHasCycle(i, -1, adj, visited)) {
21                 return true;
22             }
23         }
24         return false;
25     }
26
27     int main() {
28         int n = 4;
29         vector<vector<int>> adj(n);
30         auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
31         addEdge(0, 1); addEdge(1, 2); addEdge(2, 3); addEdge(3, 0);
32
33         cout << (hasCycleDFS(n, adj) ? "true" : "false") << endl; // true
34         return 0;
35     }

```

Approach 1 Complexity

Time. $O(V + E)$. **Space.** $O(V)$ for *visited* and the recursion stack.

Approach 2: BFS

The same parent-tracking idea, carried in the queue as a $(node, parent)$ pair instead of as a recursion argument.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool bfsHasCycle(int src, vector<vector<int>>& adj, vector<bool>& visited) {
5     queue<pair<int,int>> q; // {node, parent}
6     visited[src] = true;
7     q.push({src, -1});
8
9     while (!q.empty()) {
10         auto [node, parent] = q.front();
11         q.pop();
12
13         for (int neighbor : adj[node]) {
14             if (!visited[neighbor]) {
15                 visited[neighbor] = true;
16                 q.push({neighbor, node});
17             } else if (neighbor != parent) {
18                 return true;
19             }
20         }
21     }
22     return false;
23 }
24
25 bool hasCycleBFS(int n, vector<vector<int>>& adj) {
26     vector<bool> visited(n, false);
27     for (int i = 0; i < n; i++) {
28         if (!visited[i] && bfsHasCycle(i, adj, visited)) {
29             return true;
30         }
31     }
32     return false;
33 }
34
35 int main() {
36     int n = 4;
37     vector<vector<int>> adj(n);
38     auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
39     addEdge(0, 1); addEdge(1, 2); addEdge(2, 3); addEdge(3, 0);
40
41     cout << (hasCycleBFS(n, adj) ? "true" : "false") << endl; // true
42     return 0;
43 }

```

Dry run

Setup. Take the 4-cycle 0–1–2–3–0, DFS starting at 0.

$dfs(0, -1)$: visit 0. Neighbor 1: unvisited, recurse $dfs(1, 0)$: visit 1. Neighbor 0: visited, but 0 = *parent*, skip. Neighbor 2: unvisited, recurse $dfs(2, 1)$: visit 2. Neighbor 1: visited,

= *parent*, skip. Neighbor 3: unvisited, recurse *dfs*(3, 2): visit 3. Neighbor 2: visited, = *parent*, skip. Neighbor 0: visited, and $0 \neq \text{parent}(= 2)$ – **cycle detected**, return **true** all the way up.

Approach 2 Complexity

Time. $O(V + E)$. **Space.** $O(V)$ for *visited* and the queue.

Takeaway

Both traversals solve this with the identical idea – carry the parent along, flag any visited non-parent neighbor as a cycle – differing only in whether that extra state rides on the recursion stack (DFS) or in the queue alongside each node (BFS). This same parent-carrying trick reappears, adapted, in the Topological Sort chapter’s directed-graph cycle detection.

1.3 Bipartite Graph Check

Problem Statement. Given a graph, determine whether its nodes can be colored using exactly **two** colors such that no two adjacent nodes share the same color (such a graph is called **bipartite**).

Pattern Recognition

“Can this be split into two groups with no same-group edges” is a traversal that carries a **color** along: color the start node 0, every neighbor the **opposite** color, and so on outward. The graph is bipartite exactly when this coloring never forces a conflict – an already-colored neighbor found to share the current node’s color.

Why a conflict means *not* bipartite

If traversal reaches a node v that already has a color, and that color matches the current node u ’s color, it means two different paths from the start assigned u and v the *same* color despite u and v being directly connected – which is only possible if the cycle formed by those two paths plus the edge $u-v$ has **odd** length (an even-length cycle alternates colors consistently all the way around; an odd-length cycle cannot). So this check is, in effect, testing for the absence of odd-length cycles.

Approach 1: BFS

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool bfsCheckBipartite(int src, vector<vector<int>>& adj, vector<int>& color) {
5     queue<int> q;
6     color[src] = 0;
7     q.push(src);
8
9     while (!q.empty()) {
10         int node = q.front();
11         q.pop();
12
13         for (int neighbor : adj[node]) {
14             if (color[neighbor] == -1) {
15                 color[neighbor] = 1 - color[node];

```

```

16         q.push(neighbor);
17     } else if (color[neighbor] == color[node]) {
18         return false;
19     }
20 }
21 }
22 return true;
23 }
24
25 bool isBipartite(int n, vector<vector<int>>& adj) {
26     vector<int> color(n, -1);
27     for (int i = 0; i < n; i++) {
28         if (color[i] == -1 && !bfsCheckBipartite(i, adj, color)) {
29             return false;
30         }
31     }
32     return true;
33 }
34
35 int main() {
36     int n = 4;
37     vector<vector<int>> adj(n);
38     auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
39     addEdge(0, 1); addEdge(1, 2); addEdge(2, 3); addEdge(3, 0);
40
41     cout << (isBipartite(n, adj) ? "true" : "false") << endl; // true (even cycle)
42     return 0;
43 }

```

Approach 1 Complexity

Time. $O(V + E)$. **Space.** $O(V)$ for *color* and the queue.

Approach 2: DFS

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool dfsCheckBipartite(int node, int c, vector<vector<int>>& adj, vector<int>&
5      color) {
6      color[node] = c;
7
8      for (int neighbor : adj[node]) {
9          if (color[neighbor] == -1) {
10             if (!dfsCheckBipartite(neighbor, 1 - c, adj, color)) return false;
11         } else if (color[neighbor] == c) {
12             return false;
13         }
14     }
15     return true;
16 }
17
18 bool isBipartiteDFS(int n, vector<vector<int>>& adj) {
19     vector<int> color(n, -1);
20     for (int i = 0; i < n; i++) {
21         if (color[i] == -1 && !dfsCheckBipartite(i, 0, adj, color)) {
22             return false;
23         }
24     }
25     return true;
26 }

```

```

25 }
26
27 int main() {
28     int n = 3;
29     vector<vector<int>> adj(n);
30     auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
31     addEdge(0, 1); addEdge(1, 2); addEdge(2, 0);
32
33     cout << (isBipartiteDFS(n, adj) ? "true" : "false") << endl; // false (odd
34         cycle, triangle)
35     return 0;
36 }

```

Dry run

Setup. Take the triangle 0–1–2–0 (a 3-cycle), DFS from 0.

$dfs(0,0)$: color 0 with 0. Neighbor 1: uncolored, recurse $dfs(1,1)$: color 1 with 1. Neighbor 0: colored $0 \neq 1$, fine (it's the parent direction, naturally opposite). Neighbor 2: uncolored, recurse $dfs(2,0)$: color 2 with 0. Neighbor 1: colored $1 \neq 0$, fine. Neighbor 0: colored 0, and current color is also 0 – **conflict**, return false. The odd cycle cannot be properly 2-colored.

Approach 2 Complexity

Time. $O(V + E)$. **Space.** $O(V)$ for *color* and the recursion stack.

Takeaway

Carrying a small piece of alternating state (a color, a parity bit) through a traversal is a general technique for detecting structural properties tied to path **parity** – odd cycles here, and the same idea extends to any problem phrased as “can these be split into two groups with no same-group connections.”

1.4 Surrounded Regions

Problem Statement. Given an $m \times n$ board of 'X' and 'O', **capture** every region of 'O's that is fully **surrounded** by 'X's – flip every captured 'O' to 'X'. A region is **not** captured if any 'O' in it touches the border of the board.

Pattern Recognition

“Mark everything reachable from the boundary first, then process by elimination” – rather than trying to directly test whether each interior region is “surrounded” (which would require checking every cell of every region against the border), it is far simpler to run a traversal **only from the border's** 'O's: anything that traversal reaches is, by definition, connected to the border and therefore *safe*. Everything left unmarked afterward is unreachable from the border – the surrounded regions – and gets flipped.

Algorithm

1. For every 'O' on the board's **border** (first row, last row, first column, last column), run DFS (or BFS), marking every 4-directionally connected 'O' with a temporary marker (e.g. '#') instead of visiting a separate *visited* array.

2. Scan the entire board: any remaining '0' (never reached from the border) is surrounded – flip it to 'X'. Any '#' (reached from the border, safe) gets restored to '0'.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void markSafe(int r, int c, vector<vector<char>>& board) {
5      int rows = board.size(), cols = board[0].size();
6      if (r < 0 || r >= rows || c < 0 || c >= cols) return;
7      if (board[r][c] != '0') return;
8
9      board[r][c] = '#';
10     markSafe(r - 1, c, board);
11     markSafe(r + 1, c, board);
12     markSafe(r, c - 1, board);
13     markSafe(r, c + 1, board);
14 }
15
16 void solve(vector<vector<char>>& board) {
17     int rows = board.size(), cols = board[0].size();
18
19     for (int r = 0; r < rows; r++) {
20         markSafe(r, 0, board);
21         markSafe(r, cols - 1, board);
22     }
23     for (int c = 0; c < cols; c++) {
24         markSafe(0, c, board);
25         markSafe(rows - 1, c, board);
26     }
27
28     for (int r = 0; r < rows; r++) {
29         for (int c = 0; c < cols; c++) {
30             if (board[r][c] == '0') board[r][c] = 'X';
31             else if (board[r][c] == '#') board[r][c] = '0';
32         }
33     }
34 }
35
36 int main() {
37     vector<vector<char>> board = {
38         {'X', 'X', 'X', 'X'},
39         {'X', '0', '0', 'X'},
40         {'X', 'X', '0', 'X'},
41         {'X', '0', 'X', 'X'}
42     };
43     solve(board);
44     for (auto& row : board) {
45         for (char x : row) cout << x << " ";
46         cout << endl;
47     }
48     return 0;
49 }

```

Dry run

Setup. Take the 4×4 board above.

Border '0's: only (3,1) is on the border (last row) – but checking the full border scan, none of row 0, row 3 (except (3,1), which is '0'), column 0, or column 3 contain other '0's.

`markSafe(3, 1)` marks just $(3, 1)$ as '#' (its neighbors $(2, 1) = X$, $(3, 0) = X$, $(3, 2) = X$ are all 'X', so the safe region is just this one cell). The interior region $\{(1, 1), (1, 2), (2, 2)\}$ is never touched by any border-seeded traversal, since it doesn't connect to $(3, 1)$ (blocked by 'X' on all sides). Final scan: the interior '0's flip to 'X'; $(3, 1)$'s '#' reverts to '0'.

Complexity

Time Complexity. $O(\text{rows} \times \text{cols})$ – each cell is visited a constant number of times (once by the border traversal if reachable, plus the one final scan). **Space Complexity.** $O(\text{rows} \times \text{cols})$ recursion stack in the worst case.

Takeaway

“Process by elimination from the boundary” is a reusable trick whenever a problem’s condition is phrased in terms of *not* reaching some special set of cells (the border, here) – traverse *from* that special set first, and treat everything unreached as the answer. The next section, Number of Enclaves, reuses this exact same boundary-first elimination on a near-identical setup.

1.5 Number of Enclaves

Problem Statement. Given a binary grid where 0 is sea and 1 is land, a move walks 4-directionally from one land cell to another. Count the number of land cells from which it is **impossible** to walk off the boundary of the grid in any number of moves.

Pattern Recognition

This is Section 1.4’s boundary-first elimination technique again, with the roles renamed: land cells connected to the border can “escape,” so traverse from every **border land cell** first, marking everything reachable as escapable; every **unmarked** land cell remaining afterward is an enclave, and the answer is simply the count of those.

Algorithm

1. For every land cell on the border, DFS (or BFS) outward, marking every 4-directionally connected land cell as visited (mutate the grid in place, setting visited land to 0, exactly as the previous section reused the grid itself as the *visited* structure).
2. Scan the entire grid and count every remaining land cell (1) – these are cells no border-seeded traversal ever reached, i.e. the enclaves.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void markEscapable(int r, int c, vector<vector<int>>& grid) {
5     int rows = grid.size(), cols = grid[0].size();
6     if (r < 0 || r >= rows || c < 0 || c >= cols) return;
7     if (grid[r][c] != 1) return;
8
9     grid[r][c] = 0;
10    markEscapable(r - 1, c, grid);

```

```

11     markEscapable(r + 1, c, grid);
12     markEscapable(r, c - 1, grid);
13     markEscapable(r, c + 1, grid);
14 }
15
16 int numEnclaves(vector<vector<int>>& grid) {
17     int rows = grid.size(), cols = grid[0].size();
18
19     for (int r = 0; r < rows; r++) {
20         markEscapable(r, 0, grid);
21         markEscapable(r, cols - 1, grid);
22     }
23     for (int c = 0; c < cols; c++) {
24         markEscapable(0, c, grid);
25         markEscapable(rows - 1, c, grid);
26     }
27
28     int enclaves = 0;
29     for (int r = 0; r < rows; r++) {
30         for (int c = 0; c < cols; c++) {
31             if (grid[r][c] == 1) enclaves++;
32         }
33     }
34     return enclaves;
35 }
36
37 int main() {
38     vector<vector<int>> grid = {
39         {0, 0, 0, 0},
40         {1, 0, 1, 0},
41         {0, 1, 1, 0},
42         {0, 0, 0, 0}
43     };
44     cout << numEnclaves(grid) << endl; // 3
45     return 0;
46 }

```

Dry run

Setup. Take the 4×4 grid above.

Border land cells: $(1,0)$ (column 0). *markEscapable*(1,0): sets $(1,0)$ to 0; its only land neighbor would need to be checked, but $(0,0) = 0$, $(2,0) = 0$, $(1,1) = 0$ – no further spread. No other border cell is land. The remaining land cells $(1,2)$, $(2,1)$, $(2,2)$ form a connected interior region untouched by the border traversal – they are never adjacent to the border, and $(1,0)$'s region doesn't reach them. Final count: 3 enclaves.

Complexity

Time Complexity. $O(\text{rows} \times \text{cols})$. **Space Complexity.** $O(\text{rows} \times \text{cols})$ recursion stack in the worst case.

Takeaway

Surrounded Regions and Number of Enclaves are, structurally, the same algorithm: seed a traversal from the boundary, and treat whatever it fails to reach as the answer. Once this boundary-first elimination pattern is recognized, both problems reduce to writing the same dozen lines of code with different variable names.

1.6 Word Ladder I

Problem Statement. Given a *beginWord*, an *endWord*, and a *wordList*, find the length of the **shortest** transformation sequence from *beginWord* to *endWord*, changing exactly one letter at a time, where every intermediate word must exist in *wordList*. Return 0 if no such sequence exists.

Pattern Recognition

“Shortest sequence of single-character transformations” is shortest-path-in-an-unweighted-graph – exactly what BFS computes – but the graph is entirely **implicit**: there is no adjacency list. A word’s neighbors are generated on the fly by trying every possible single-character substitution at every position and checking whether the result is a valid word in the list.

Generating neighbors on the fly

For a word of length L using a 26-letter alphabet, there are $26L$ candidate single-character substitutions to try at each step. Each candidate is checked against a hash set of valid words (built from *wordList*) in $O(L)$ (for the hash and string comparison) – so generating all neighbors of one word costs $O(26L^2)$, in place of looking up a precomputed adjacency list.

Algorithm

1. Build a hash set from *wordList* for $O(1)$ -ish membership checks (and remove words from the set as they are used, which serves as the *visited* marker).
2. BFS from *beginWord*, tracking the transformation count alongside each word in the queue.
3. For each word popped, try every single-character substitution at every position; if the result is in the set, it’s a valid next step – remove it from the set (mark visited) and push it with count $+1$. If the result equals *endWord*, return the count $+1$ immediately.
4. If the queue empties without reaching *endWord*, return 0.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
5      unordered_set<string> words(wordList.begin(), wordList.end());
6      if (words.find(endWord) == words.end()) return 0;
7
8      queue<pair<string,int>> q;
9      q.push({beginWord, 1});
10     words.erase(beginWord);
11
12     while (!q.empty()) {
13         auto [word, steps] = q.front();
14         q.pop();
15
16         if (word == endWord) return steps;
17
18         for (int i = 0; i < (int)word.size(); i++) {
19             char original = word[i];
20             for (char c = 'a'; c <= 'z'; c++) {

```

```

21         if (c == original) continue;
22         word[i] = c;
23         if (words.find(word) != words.end()) {
24             words.erase(word);
25             q.push({word, steps + 1});
26         }
27     }
28     word[i] = original;
29 }
30 }
31 return 0;
32 }
33
34 int main() {
35     string beginWord = "hit", endWord = "cog";
36     vector<string> wordList = {"hot", "dot", "dog", "lot", "log", "cog"};
37     cout << ladderLength(beginWord, endWord, wordList) << endl; // 5
38     return 0;
39 }

```

Dry run

Setup. Take $beginWord = \text{"hit"}$, $endWord = \text{"cog"}$, $wordList = \{\text{hot}, \text{dot}, \text{dog}, \text{lot}, \text{log}, \text{cog}\}$.

Queue: $[(\text{hit}, 1)]$. Pop hit : trying all substitutions, hot matches (set has it); push $(\text{hot}, 2)$, erase hot . Pop hot : substitutions dot and lot both match; push $(\text{dot}, 3)$, $(\text{lot}, 3)$. Pop dot : dog matches; push $(\text{dog}, 4)$. Pop lot : log matches; push $(\text{log}, 4)$. Pop dog : cog matches; push $(\text{cog}, 5)$. Pop log : cog already erased from the set (claimed by dog 's expansion), nothing new. Pop cog : word = $endWord$, return 5.

Complexity

Time Complexity. $O(N \cdot 26L^2)$, where N is the number of words and L is word length – each of up to N words is popped once, and generating + checking its $26L$ candidate substitutions costs $O(26L^2)$ (the L factor from string copying/hashing each candidate).

Space Complexity. $O(N \cdot L)$ for the word set and the queue.

Takeaway

When a problem's "graph" has no explicit edge list and instead defines adjacency through some transformation rule (here, single-character substitution), generate neighbors on demand inside the BFS loop rather than trying to materialize the whole graph up front – the BFS structure itself is completely unchanged from the explicit-adjacency-list case.

1.7 Word Ladder II

Problem Statement. Given $beginWord$, $endWord$, and $wordList$, return **every** shortest transformation sequence from $beginWord$ to $endWord$ (not just the length, as in Section 1.6).

Pattern Recognition

"Return every shortest path, not just its length" needs the same BFS as Word Ladder I, but with one critical addition: a **parent map** recording, for every word, every word at the *previous level* that could reach it – the same "print the path" machinery used by reconstruction problems throughout this book series (e.g. the DP on LIS chapter's

Print LIS): *measuring* an optimum and *reconstructing* every optimal path are different problems, and the second always needs extra bookkeeping kept during the first.

The same-level subtlety

A naive BFS removes a word from the “available” set the instant it is reached, to avoid revisiting it. But for **counting all shortest paths**, two different words at the *same* level might both be able to reach the same word at the *next* level – both of those connections are valid parts of some shortest path and must be recorded. So word removal must be **deferred until an entire level finishes processing**: only after every word at the current level has tried every substitution should the newly-discovered next-level words be removed from the available set.

Algorithm

1. BFS level by level. For each word in the **current** level, generate every single-character substitution; for each one found in the available word set, record the current word as one of its parents, and collect it into the **next** level’s word set (without removing it from the global available set yet).
2. After the whole current level is processed, remove every newly-collected next-level word from the global available set (the deferred removal), then advance to the next level.
3. Stop as soon as *endWord* is found among the newly-collected words for some level (the shortest distance is confirmed, and no longer-path could still be shortest).
4. Backtrack from *endWord* via the parent map, using DFS, collecting every path back to *beginWord*, then reverse each collected path.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void backtrack(string word, string& beginWord,
5               unordered_map<string, vector<string>>& parents,
6               vector<string>& path, vector<vector<string>>& result) {
7     path.push_back(word);
8     if (word == beginWord) {
9         vector<string> reversed(path.rbegin(), path.rend());
10        result.push_back(reversed);
11    } else {
12        for (string& parent : parents[word]) {
13            backtrack(parent, beginWord, parents, path, result);
14        }
15    }
16    path.pop_back();
17 }
18
19 vector<vector<string>> findLadders(string beginWord, string endWord,
20                                vector<string>& wordList) {
21     unordered_set<string> available(wordList.begin(), wordList.end());
22     if (available.find(endWord) == available.end()) return {};
23
24     unordered_map<string, vector<string>> parents;
25     unordered_set<string> currentLevel = {beginWord};
26     available.erase(beginWord);

```

```

27     bool found = false;
28
29     while (!currentLevel.empty() && !found) {
30         unordered_set<string> nextLevel;
31
32         for (const string& word : currentLevel) {
33             string candidate = word;
34             for (int i = 0; i < (int)candidate.size(); i++) {
35                 char original = candidate[i];
36                 for (char c = 'a'; c <= 'z'; c++) {
37                     if (c == original) continue;
38                     candidate[i] = c;
39                     if (available.find(candidate) != available.end()) {
40                         nextLevel.insert(candidate);
41                         parents[candidate].push_back(word);
42                         if (candidate == endWord) found = true;
43                     }
44                 }
45                 candidate[i] = original;
46             }
47         }
48         for (const string& word : nextLevel) available.erase(word);
49         currentLevel = nextLevel;
50     }
51
52     vector<vector<string>> result;
53     if (found) {
54         vector<string> path;
55         backtrack(endWord, beginWord, parents, path, result);
56     }
57     return result;
58 }
59
60 int main() {
61     string beginWord = "hit", endWord = "cog";
62     vector<string> wordList = {"hot", "dot", "dog", "lot", "log", "cog"};
63     vector<vector<string>> ladders = findLadders(beginWord, endWord, wordList);
64     for (auto& path : ladders) {
65         for (string& w : path) cout << w << " ";
66         cout << endl;
67     }
68     // hit hot dot dog cog
69     // hit hot lot log cog
70     return 0;
71 }

```

Dry run

Setup. Same input as Section 1.6's example.

Level 0: {hit}. Level 1: {hot} (hit's only valid substitution), $parents[hot] = [hit]$. Level 2: from hot, both dot and lot match – collected together as {dot, lot}, $parents[dot] = [hot]$, $parents[lot] = [hot]$ (neither is removed from *available* until *both* are found, which is exactly the deferred-removal rule). Level 3: from dot, dog matches; from lot, log matches – {dog, log}. Level 4: from dog, cog matches (*found* = true); from log, cog *also* matches (still available, since removal happens only at the end of the level) – $parents[cog] = [dog, log]$. Backtracking from cog: two branches, through dog and through log, both leading back to hit – exactly the two shortest ladders the code prints.

Complexity

Time Complexity. $O(N \cdot 26L^2)$ for the level-by-level BFS (same bound as Word Ladder I), plus time proportional to the total length of all reconstructed paths for the backtracking step. **Space Complexity.** $O(N \cdot L)$ for the parent map and word sets, plus the space needed to store every returned path.

Takeaway

This chapter closes on the same lesson Print LIS and Print LCS taught earlier in this book, now applied to an implicit graph: finding a shortest distance and enumerating *every* path that achieves it are different problems, and the second one is solved by recording parent relationships *during* the same traversal that finds the distance – with one added wrinkle here, that ties at the same BFS level require deferring when those parent relationships are finalized.